Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №4
по дисциплине
«Современные системы программирования»

Выполнила: ст. гр. 320604
К.А. Хаджинова

Проверила: Ю.О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 ХОД РАБОТЫ 4
2.1 Постановка задачи 4
2.2 Теоретические сведения 4
2.3 Подготовка среды 5
2.4 Базовый код 5
2.5 Задание 1. Исследование утечек памяти 6
2.6 Задание 2. Исследование *LOH* 7
2.7 Задание 3. Оптимизация упаковки 7
2.8 Задание 4. Поиск и исправление утечек 8
2.9 Профилирование: *Call Tree* и горячие пути 9
2.10 Профилирование собственного кода 12
3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 14
ВЫВОДЫ 15
ПРИЛОЖЕНИЕ А (ОБЯЗАТЕЛЬНОЕ) ЛИСТИНГИ ПРОГРАММНОГО КОДА 16

1 ЦЕЛЬ РАБОТЫ

Изучить механизмы сборки мусора в *.NET*, работу с поколениями памяти, выявление утечек, анализ кучи больших объектов (*LOH*) и оптимизацию через устранение упаковки (*boxing*). Освоить профилирование производительности: построение дерева вызовов (*Call Tree*), поиск горячих путей и оценку собственного времени методов.

2 ХОД РАБОТЫ

Работа выполнена в консольном приложении *C#* для платформы *.NET* 10 (*SDK* 10.0.401), замеры выполнены в режиме *Release*. Полный текст программы приведен в приложении А. Ниже последовательно описаны постановка задачи, теоретическая основа, реализация заданий и результаты проверки.

2.1 Постановка задачи

Для поиска утечек памяти и профилирования в рамках работы требуется выполнить следующие задачи:

1 исследовать утечки памяти через статические ссылки и неотписанные события;

2 исследовать кучу больших объектов (*LOH*);

3 оптимизировать код устранением упаковки и сравнить результат;

4 найти и исправить утечки в предложенном проблемном коде;

5 выполнить профилирование: построить дерево вызовов, найти горячие пути и проанализировать собственный код.

2.2 Теоретические сведения

Сборка мусора в *.NET* работает с поколениями: *Gen0* содержит молодые объекты, *Gen1* – промежуточные, *Gen2* – долгоживущие. Объекты размером более 85 000 байт попадают в *Large Object Heap* (*LOH*). Принудительная сборка выполняется командами *GC.Collect*() и *GC.WaitForPendingFinalizers*().

Основные виды утечек памяти: статические ссылки, которые живут весь цикл приложения, неотписанные события и делегаты, неправильная работа с *IDisposable*. Упаковка (*boxing*) – преобразование значимого типа в ссылочный, она замедляет код примерно в 2–4 раза и увеличивает расход памяти.

Профилирование производительности определяет, на какие методы приходится основное время выполнения. Дерево вызовов (*Call Tree*) показывает время каждого метода и его долю от родительского метода, цепочка методов с наибольшей долей времени называется горячим путем (*Hot Path*). Время метода без учета вложенных вызовов называют собственным временем (*Own Time*).

2.3 Подготовка среды

Приложение *Lab4* создано как консольный проект *.NET* с целевой платформой *net10.0* (*SDK* 10.0.401), замеры выполнены в режиме *Release*. Режимы запуска: без аргументов – базовый код и задания 1–3, аргумент *buggy* – задание 4, аргумент *calltree* – профилирование. Время и память выводятся средствами *Stopwatch* и *GC*.

2.4 Базовый код

Базовый код методических указаний содержит классы *MemoryLeakExample*, *EventLeakExample*, *MemoryMonitor* и *StaticReferenceDemo* и демонстрирует статические ссылки, выделение памяти, утечки, упаковку, очистку и поколения объектов. Результат запуска показан на рисунках 2.1 и 2.2.

```
> dotnet Lab4.dll
=== Лабораторная работа: Управление памятью ===

1. Статические ссылки:
Объект 1: Статический [100,200], Экземплярный [1]
Объект 2: Статический [100,200], Экземплярный [2]
Статический список ОБЩИЙ для всех объектов!

2. Выделение памяти и LOH:

=== После выделения ===
Total Memory: 14 MB
Gen0: 1, Gen1: 1, Gen2: 1
Working Set: 26 MB

3. Тест утечек:

=== После утечек ===
```

Рисунок 2.1 – Результат запуска базового кода (начало)

Статический список общий для обоих объектов, экземплярные списки разные. После выделения памяти и объектов *LOH* занято 14 МБ, после создания утечек 24 МБ.

Продолжение вывода базового кода показано на рисунке 2.2.

```
Total Memory: 24 MB
Gen0: 3, Gen1: 2, Gen2: 1
Working Set: 35 MB

4. Тест упаковки:
С упаковкой: 87ms
Без упаковки: 6ms

5. После очистки:

=== Очищено ===
Total Memory: 4 MB
Gen0: 10, Gen1: 7, Gen2: 4
Working Set: 43 MB

6. Поколения объектов:
Начальное поколение: 0
После GC: 1
```

Рисунок 2.2 – Результат запуска базового кода (продолжение)

Добавление миллиона чисел с упаковкой заняло 87 мс, без упаковки 6 мс. После очистки объем памяти уменьшился до 4 МБ, а новый объект после сборки перешел из поколения 0 в поколение 1.

2.5 Задание 1. Исследование утечек памяти

В пункте 1.1 класс *StaticLeak* добавляет в статический список 1000 объектов по 10 КБ, затем вызывается *GC.Collect*(). В пункте 1.2 класс *EventLeak* подписывает 100 обработчиков, каждый из которых удерживает массив 100 КБ, после чего выполняется отписка и сборка. Код приведен в приложении А.

Результат показан на рисунке 2.3.

```
> dotnet Lab4.dll
=== Задание 1.1: статическая утечка ===
До: 307 KB
После добавления 1000 x 10KB: 10340 KB
После GC.Collect() (память не освобождается): 10328 KB

=== Задание 1.2: событийная утечка ===
До подписок: 10336 KB
После 100 подписок: 20358 KB
После GC.Collect() (подписки живы): 20343 KB
После отписки и GC.Collect(): 10339 KB
```

Рисунок 2.3 – Результат выполнения задания 1

После добавления 1000 объектов память выросла с 307 КБ до 10 340 КБ и после *GC.Collect*() осталась 10 328 КБ. После 100 подписок память выросла

с 10 336 КБ до 20 358 КБ, сборка ее не освободила, а после отписки память вернулась к 10 339 КБ.

2.6 Задание 2. Исследование *LOH*

Создаются массивы 80 000 и 90 000 байт, определяется их поколение, затем создаются 100 объектов по 100 КБ, измеряется память до и после, очищается список и вызывается *GC.Collect*(2). Результат показан на рисунке 2.4.

```
> dotnet Lab4.dll
=== Задание 2.1: создание объектов в LOH ===
small: Gen0
large: Gen2

=== Задание 2.2: мониторинг LOH ===
Память до: 10513 KB
Память после создания 100 x 100KB: 20510 KB
Память после очистки списка и GC.Collect(2): 10431 KB
```

Рисунок 2.4 – Результат выполнения задания 2

Массив 80 000 байт находится в *Gen0*, массив 90 000 байт сразу в *Gen2*, то есть в *LOH*. Память выросла с 10 513 КБ до 20 510 КБ и после очистки списка и *GC.Collect*(2) уменьшилась до 10 431 КБ.

2.7 Задание 3. Оптимизация упаковки

В пункте 3.1 сравнивается добавление миллиона чисел в *List<object>* и *List<int>*. В пункте 3.2 класс *UnoptimizedCode* на основе *ArrayList* заменен классом *OptimizedCode* на основе *List<int>*. Код приведен в приложении А.

Результат показан на рисунке 2.5.

```
> dotnet Lab4.dll
=== Задание 3.1: boxing ===
С упаковкой (List<object>): 105 мс, +31650 KB
Без упаковки (List<int>): 3 мс, +8200 KB

=== Задание 3.2: UnoptimizedCode и OptimizedCode ===
UnoptimizedCode (ArrayList): 95 мс, +37774 KB
OptimizedCode (List<int>): 4 мс, +8202 KB
```

Рисунок 2.5 – Результат выполнения задания 3

Вариант *List<object>* выполнился за 105 мс и занял 31 650 КБ, *List<int>* – за 3 мс и 8 200 КБ. Для *ArrayList* получено 95 мс и 37 774 КБ, для *List<int>* – 4 мс и 8 202 КБ. Причина разницы в том, что при упаковке каждое число становится отдельным объектом в куче.

2.8 Задание 4. Поиск и исправление утечек

В классе *BuggyCode* найдены три источника утечек. Описание и исправление приведены в таблице 3.1.

Данные по теме «утечки в классе *BuggyCode*» приведены в таблице 2.1.

Таблица 2.1 – Утечки в классе *BuggyCode*

Место	Причина	Исправление
_globalEvent += OnEvent	Статическое событие хранит ссылку на каждый объект, подписка не снимается	Отписка в методе *Dispose*
_cache.Add(*new MemoryStream*(1024))	Статический кэш хранит потоки и не освобождает их	Удаление из кэша и вызов *Dispose* у потока
Byte[] *_data* (1 МБ)	Массив достижим через подписку и не собирается	Освобождается после отписки

Исправленный класс реализует *IDisposable*. Код приведен в приложении А.

Результат запуска для 100 объектов показан на рисунке 2.6.

```
> dotnet Lab4.dll buggy

=== Задание 4: код с утечками (BuggyCode, 100 объектов) ===
Память до: 0 MB
Память после создания и GC.Collect(): 100 MB
Объектов в статическом кэше: 100

=== Задание 4: исправленная версия (FixedCode, 100 объектов) ===
Память до: 100 MB
Память после создания, Dispose и GC.Collect(): 100 MB
Объектов в статическом кэше: 0
```

Рисунок 2.6 – Результат выполнения задания 4

Для *BuggyCode* после сборки мусора память составила 100 МБ, а в статическом кэше осталось 100 объектов. При использовании *FixedCode* с вызовом *Dispose* в кэше объектов нет, а рост памяти прекратился (показание 100 МБ осталось прежним).

2.9 Профилирование: *Call Tree* и горячие пути

Программа из методических указаний генерирует 10 000 строк журнала и 100 аномальных записей, фильтрует их и анализирует метод *IsAnomaly*. Время каждого шага накапливается в словаре и выводится в виде дерева вызовов с долями от родительского метода. В примере из методических указаний дата берется из *parts*[1], где находится уровень журнала (*INFO*, *ERROR*), поэтому *DateTime.TryParse* всегда возвращает *false*, метод до *Regex* не доходит, а вывод дерева завершается ошибкой отсутствия ключа *IsAnomaly.Checks* (рисунок 2.7).

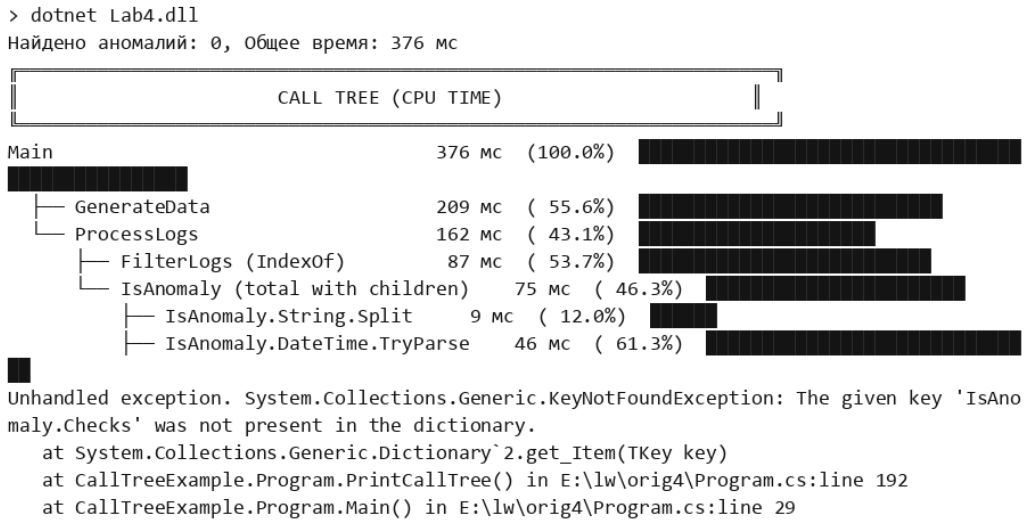


Рисунок 2.7 – Запуск кода из методических указаний без изменений

Исходный код находит 0 аномалий, выводит часть дерева вызовов и завершается исключением *KeyNotFoundException*, так как метод *IsAnomaly.Checks* ни разу не вызывался.

В выполненной работе дата берется из *parts*[0] после префикса *USERnn*:, время хранится как *double* (долей миллисекунды). Перед замерами выполняется один прогон без вывода для прогрева *JIT*. Фрагмент разбора даты приведен в приложении А.

Дерево вызовов с *DateTime.TryParse* показано на рисунках 2.8 и 2.9.

```
> dotnet Lab4.dll calltree
=== Call Tree: DateTime.TryParse ===

Найдено аномалий: 404, Общее время: 41 мс

================== CALL TREE (CPU TIME) ==================

Main                                          41.1 мс  (100.0%)  ##############################
###################
  |-- GenerateData                            27.2 мс  ( 66.1%)  ##############################
##
  `-- ProcessLogs                             13.9 мс  ( 33.8%)  ################
      |-- FilterLogs (IndexOf)                 0.7 мс  (  4.9%)  ##
      `-- IsAnomaly (total with children)     13.2 мс  ( 95.1%)  ##############################
################
          |-- IsAnomaly.String.Split           0.9 мс  (  7.1%)  ###
          |-- IsAnomaly.DateTime.TryParse      2.5 мс  ( 19.1%)  #########
          |-- IsAnomaly.Regex.Match            2.9 мс  ( 21.9%)  ##########
```

Рисунок 2.8 – *Call Tree* с *DateTime.TryParse* (начало)

Общее время выполнения составило 41 мс. На *GenerateData* приходится 27,2 мс (66 %), на *ProcessLogs* – 13,9 мс, из которых 13,2 мс занимает *IsAnomaly*.

Статистика вызовов показана на рисунке 2.9.

```
          |-- IsAnomaly.Checks                 1.4 мс  ( 10.8%)  #####
          `-- IsAnomaly (own time)            12.4 мс  ( 93.7%)  ##############################
###############

================== Горячий путь (Hot Path) ==================
Main -> ProcessLogs -> IsAnomaly -> Regex.Match

Статистика вызовов:
   GenerateData: 1 вызовов, 27.2 мс
   FilterLogs (IndexOf): 1 вызовов, 0.7 мс
   IsAnomaly.String.Split: 4975 вызовов, 0.9 мс
   IsAnomaly.DateTime.TryParse: 4975 вызовов, 2.5 мс
   IsAnomaly.Regex.Match: 4975 вызовов, 2.9 мс
   IsAnomaly.Checks: 4975 вызовов, 1.4 мс
   IsAnomaly (own time): 4975 вызовов, 12.4 мс
   IsAnomaly.CheckLength: 100 вызовов, 0.0 мс
   IsAnomaly (total with children): 1 вызовов, 13.2 мс
   ProcessLogs: 1 вызовов, 13.9 мс
```

Рисунок 2.9 – *Call Tree* с *DateTime.TryParse* (продолжение)

Каждый шаг *IsAnomaly* вызывался 4975 раз. Горячий путь: *Main*, *ProcessLogs*, *IsAnomaly*, *Regex.Match*.

Методы с наибольшим собственным временем сведены в таблице 3.2.

Сведения по теме «методы с наибольшим собственным временем» сведены в таблицу 2.2.

Таблица 2.2 – Методы с наибольшим собственным временем (*DateTime.TryParse*)

Метод	Время, мс	Доля от родителя, %	Причина
GenerateData	27,2	66,1	Формирование 10 000 строк: форматирование дат, *StringBuilder*, генерация случайных чисел
IsAnomaly.Regex.Match	2,9	21,9	Сопоставление регулярного выражения для каждой строки
IsAnomaly.DateTime.TryParse	2,5	19,1	Разбор даты с учетом культуры
IsAnomaly.Checks	1,4	10,8	Поиск подстроки и сравнение дат
IsAnomaly.String.Split	0,9	7,1	Создание массива строк для каждой записи

Замена *DateTime.TryParse* на *DateTime.ParseExact* с фиксированным форматом *yyyy-MM-dd HH*:*mm*:*ss.fff* выполнена в том же методе. Дерево вызовов показано на рисунках 2.10 и 2.11.

```
> dotnet Lab4.dll calltree
=== Call Tree: DateTime.ParseExact ===

Найдено аномалий: 395, Общее время: 48 мс

=================== CALL TREE (CPU TIME) ====================

Main                                          48.2 мс  (100.0%)  ##############################
###################
  |-- GenerateData                            23.9 мс  ( 49.6%)  ########################
  `-- ProcessLogs                             24.3 мс  ( 50.4%)  ########################
      |-- FilterLogs (IndexOf)                 0.7 мс  (  3.1%)  #
      `-- IsAnomaly (total with children)     23.5 мс  ( 96.9%)  ##############################
#################
          |-- IsAnomaly.String.Split           1.3 мс  (  5.4%)  ##
          |-- IsAnomaly.DateTime.ParseExact    7.1 мс  ( 30.4%)  ###############
          |-- IsAnomaly.Regex.Match            6.3 мс  ( 27.0%)  #############
          |-- IsAnomaly.Checks                 2.3 мс  (  9.9%)  ####
```

Рисунок 2.10 – *Call Tree* с *DateTime.ParseExact* (начало)

Общее время составило 48 мс, из них *IsAnomaly* занимает 23,5 мс. Разбор даты *ParseExact* занял 7,1 мс (30,4 %), *Regex* – 6,3 мс (27,0 %).

Продолжение вывода показано на рисунке 2.11.

```
         `-- IsAnomaly (own time)               22.3 мс  ( 94.7%)  ##############################
################

=================== Горячий путь (Hot Path) ===================
Main -> ProcessLogs -> IsAnomaly -> IsAnomaly.DateTime.ParseExact

Статистика вызовов:
   GenerateData: 1 вызовов, 23.9 мс
   FilterLogs (IndexOf): 1 вызовов, 0.7 мс
   IsAnomaly.String.Split: 4945 вызовов, 1.3 мс
   IsAnomaly.DateTime.ParseExact: 4945 вызовов, 7.1 мс
   IsAnomaly.Regex.Match: 4945 вызовов, 6.3 мс
   IsAnomaly.Checks: 4945 вызовов, 2.3 мс
   IsAnomaly (own time): 4945 вызовов, 22.3 мс
   IsAnomaly.CheckLength: 100 вызовов, 0.0 мс
   IsAnomaly (total with children): 1 вызовов, 23.5 мс
   ProcessLogs: 1 вызовов, 24.3 мс
```

Рисунок 2.11 – *Call Tree* с *DateTime.ParseExact* (продолжение)

Число вызовов каждого шага 4945, горячим путем стал разбор даты. В выполненном замере *ParseExact* оказался медленнее *TryParse* (7,1 мс против 2,5 мс), то есть заметного выигрыша от замены нет. *ParseExact* оправдан, когда формат строго задан и строки в других форматах нужно отвергать.

2.10 Профилирование собственного кода

Для профилирования написан небольшой код обработки текста: сборка строки из 20 000 слов конкатенацией в цикле, сборка такой же строки через *StringBuilder*, подсчет слов и сортировка 200 000 чисел. Время каждого метода измеряется *Stopwatch*. Код основных методов приведен в приложении А.

Результат показан на рисунке 2.12.

```
> dotnet Lab4.dll calltree
=== Профилирование собственного кода ===

Длина текстов: 120000 и 120000, слов: 20000, чисел: 200000

Общее время: 595.2 мс

Метод                    Вызовов   Время, мс   Доля, %
BuildTextWithConcat            1       575.0      96.6
SortNumbers                    1        17.9       3.0
CountWords                     1         0.8       0.1
BuildTextWithBuilder           1         0.8       0.1
```

Рисунок 2.12 – Профиль собственного кода

Общее время составило 595 мс. Метод *BuildTextWithConcat* занял 575,0 мс (96,6 %), тогда как *BuildTextWithBuilder* – 0,8 мс. Причина в том, что при конкатенации строк в цикле на каждом шаге создается новая строка и

копируется весь накопленный текст, поэтому затраты растут квадратично. Горячий путь собственного кода – метод конкатенации.

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Почему объекты в статическом списке не удаляются?

Статическое поле является корнем для сборщика мусора и живет до завершения приложения. Все объекты, достижимые из статического списка, считаются используемыми, поэтому *GC.Collect*() их не освобождает.

2. Почему события вызывают утечки?

Событие хранит ссылки на делегаты подписчиков. Пока подписчик не отписался, источник события удерживает его и все, на что ссылается обработчик. Если источник живет долго, подписчики не освобождаются. После отписки (оператор -=) ссылка исчезает, и память освобождается.

3. Почему объекты больше 85 КБ попадают в *LOH* и почему *LOH* не уплотняется?

Для больших объектов используется отдельная куча, чтобы не копировать большие блоки при уплотнении поколений. Из-за стоимости копирования *LOH* обычно не уплотняется, поэтому освобожденные места остаются фрагментированными.

4. Какие виды утечек существуют и что такое горячий путь?

Основные виды утечек: статические ссылки, неотписанные события и делегаты, неосвобожденные *IDisposable*-объекты, зацикленные ссылки. Горячий путь – цепочка вызовов, на которую приходится наибольшая доля времени выполнения. В выполненной работе горячим путем в дереве вызовов был разбор строки (регулярное выражение и разбор даты), а в собственном коде – конкатенация строк в цикле. Оптимизировать следует именно такие участки.

ВЫВОДЫ

Изучены механизмы сборки мусора, поколения и *LOH*. Показано, что статическая коллекция и неотписанные события удерживают память после *GC.Collect*(), а отписка и очистка ее освобождают. Объекты размером более 85 000 байт сразу размещаются в *Gen2*. Устранение упаковки сократило время добавления миллиона элементов до единиц миллисекунд. Найдены три источника утечек в классе *BuggyCode* и написана исправленная версия. По дереву вызовов определены методы с наибольшим собственным временем и горячие пути, в собственном коде выявлена конкатенация строк, занимающая 97 % времени. Замена *TryParse* на *ParseExact* ускорения в замере не дала. Поставленная цель достигнута.

ПРИЛОЖЕНИЕ А
(ОБЯЗАТЕЛЬНОЕ)
ЛИСТИНГИ ПРОГРАММНОГО КОДА

Листинг А.1 – Классы *StaticLeak* и *EventLeak* (задание 1)

```
// Класс со статическим списком (1.1)
public class StaticLeak
{
    private static List<byte[]> _list = new List<byte[]>();

    public static void AddData()
    {
        for (int i = 0; i < 1000; i++)
            _list.Add(new byte[1024 * 10]);
    }
}

// Класс с событием (1.2): большой объект создаётся при подписке и удерживается обработчиком
public class EventLeak
{
    public event EventHandler? BigEvent;
    private List<EventHandler> _handlers = new List<EventHandler>();

    public void Subscribe()
    {
        var data = new byte[1024 * 100];
        EventHandler handler = (sender, e) => { data[0]++; };
        BigEvent += handler;
        _handlers.Add(handler);
    }

    public void UnsubscribeAll()
    {
        foreach (var handler in _handlers)
            BigEvent -= handler;
        _handlers.Clear();
    }
}
```

Листинг А.2 – Классы *UnoptimizedCode* и *OptimizedCode* (задание 3.2)

```
// НЕОПТИМИЗИРОВАННО
public class UnoptimizedCode
{
    private ArrayList _data = new ArrayList();
    public void Add(int value) => _data.Add(value); // Boxing!
}

// ОПТИМИЗИРОВАННО
public class OptimizedCode
```

```
{
    private List<int> _data = new List<int>();
    public void Add(int value) => _data.Add(value); // Без boxing
}
```

Листинг А.3 – Классы *BuggyCode* и *FixedCode*

```
public class BuggyCode
{
    private static List<IDisposable> _cache = new List<IDisposable>();
    private static event EventHandler? _globalEvent;
    private byte[] _data = new byte[1024 * 1024];

    public BuggyCode()
    {
        _globalEvent += OnEvent;            // Утечка 1: подписка на статическое событие без отписки
        _cache.Add(new MemoryStream(1024));   // Утечка 2: статический кэш хранит MemoryStream и не освобождает его
    }

    private void OnEvent(object? sender, EventArgs e)
    {
        var temp = new byte[1024 * 100];
    }

    public static int CacheCount => _cache.Count;
}

public class FixedCode : IDisposable
{
    private static List<IDisposable> _cache = new List<IDisposable>();
    private static event EventHandler? _globalEvent;
    private byte[]? _data = new byte[1024 * 1024];
    private MemoryStream _stream = new MemoryStream(1024);

    public FixedCode()
    {
        _globalEvent += OnEvent;
        _cache.Add(_stream);
    }

    private void OnEvent(object? sender, EventArgs e)
    {
        var temp = new byte[1024 * 100];
    }

    public void Dispose()
    {
        _globalEvent -= OnEvent;   // отписка от события
        _cache.Remove(_stream);    // удаление из статического кэша
        _stream.Dispose();         // освобождение потока
        _data = null;
```

```
    }

    public static int CacheCount => _cache.Count;
}
```

Листинг А.4 – Фрагмент метода *IsAnomaly*

```
// Шаг 3: разбор даты (тяжелый). Дата находится в первой части строки после
"USERnn:"
string dateText = parts[0].Substring(parts[0].IndexOf(':')  + 1).Trim();
var dateSw = Stopwatch.StartNew();
DateTime logDate = default;
bool dateOk;
if (_parseExact)
{
    // Задание 2: DateTime.ParseExact с фиксированным форматом
    try
    {
        logDate = DateTime.ParseExact(dateText, "yyyy-MM-dd HH:mm:ss.fff",
CultureInfo.InvariantCulture);
        dateOk = true;
    }
    catch (FormatException)
    {
        dateOk = false;
    }
}
else
{
    dateOk = DateTime.TryParse(dateText, out logDate);
}
dateSw.Stop();
AddCall(_parseExact ? "IsAnomaly.DateTime.ParseExact" : "IsAnomaly.DateTime.TryParse",
dateSw.Elapsed.TotalMilliseconds);
```

Листинг А.5 – Профилируемые методы

```
// Конкатенация строк в цикле: каждый раз создаётся новая строка
private static string BuildTextWithConcat(int count)
{
    var sw = Stopwatch.StartNew();
    string result = "";
    for (int i = 0; i < count; i++)
        result += "слово ";
    Add("BuildTextWithConcat", sw);
    return result;
}

private static string BuildTextWithBuilder(int count)
{
    var sw = Stopwatch.StartNew();
    var sb = new StringBuilder();
    for (int i = 0; i < count; i++)
```

```
        sb.Append("слово ");
    Add("BuildTextWithBuilder",  sw);
    return sb.ToString();
}
```